

CheckIt.AI — Pipeline de transformation et schéma de données

Étape 03 — Transformer, contrôler et préparer le chargement

État du projet vérifié le 2 septembre 2026

1. Objectif et grain des données

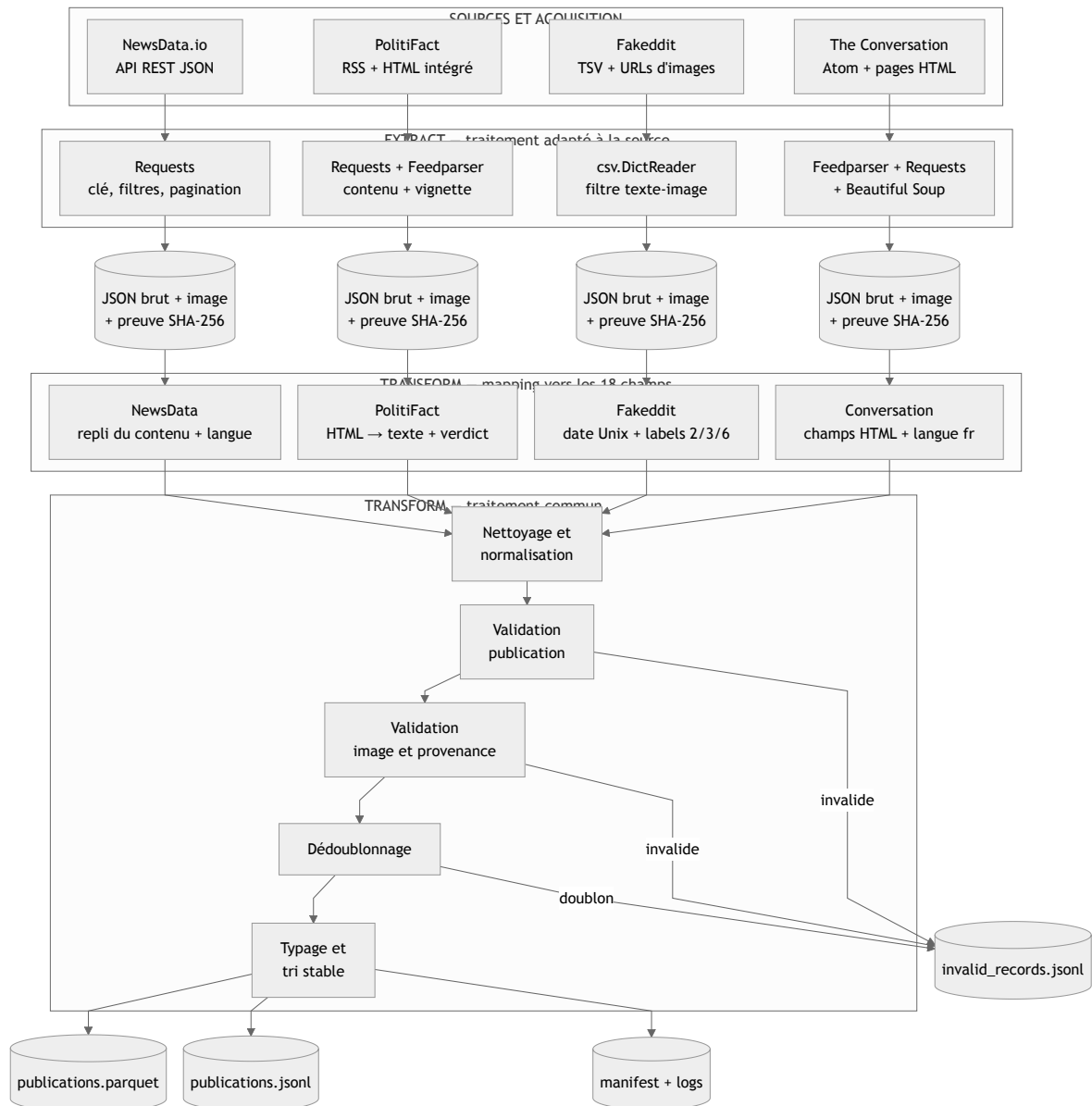
Cette étape transforme les lots bruts des quatre extracteurs en un contrat commun, reproductible et journalisé. Le fichier principal chargé par Airflow dans PostgreSQL est `data/processed/publications.parquet`.

Son grain est : **une ligne par publication unique, associée à exactement une image locale validée.**

- **clé primaire logique** : `publication_id` ;
- **nombre de colonnes** : 18 ;
- **image** : obligatoire et stockée sur la même ligne que son texte ;
- **label** : facultatif, conservé sans réinterprétation ;
- **dédoublonnage secondaire** : `source_url` + `image_sha256`.

Le schéma conceptuel reprend volontairement cette structure plate. Il correspond à la table PostgreSQL réellement alimentée par le pipeline, sans ajouter d'entités inutiles.

2. Pipeline de l'acquisition à la transformation



3. Traitement propre à chaque source

Source	Acquisition	Mapping vers le contrat commun
NewsData.io	API REST avec Requests, clé d'environnement, filtres, pagination et retries	article_id devient newdata_<id> ; le texte suit le repli content → description → ai_summary ; french devient fr
PolitiFact	RSS téléchargé avec Requests, puis lu avec Feedparser	SHA-256 de l'ID RSS ou de l'URL sur 16 caractères ; BeautifulSoup nettoie content_html et lit le verdict du flat-meter
Fakeddit	TSV lu ligne par ligne avec csv.DictReader ; seules les lignes texte-image exploitables sont retenues	ID Reddit préfixé par fakeddit_ ; date Unix vers UTC ; labels 2/3/6 regroupés dans une chaîne JSON
The Conversation France	Flux Atom pour les URLs, puis page téléchargée et analysée avec BeautifulSoup	ID numérique de l'article, ou hash stable de l'URL ; texte HTML déjà extrait ; langue fixée à fr ; aucun label inventé

Les quatre fonctions `transform_<source>()` produisent le même dictionnaire provisoire. Le traitement commun peut alors appliquer les mêmes validations à toutes les sources.

4. Traitement commun, dédoublonnage et journalisation

Le script `scripts/transform_data.py` suit quatre étapes visibles dans `logs/transform.log` :

1. **Lecture** : contrôle de chaque enveloppe JSON, de sa source, de sa date de collecte et du nombre d'enregistrements.
2. **Mapping et validation** : nettoyage Unicode et espaces, dates UTC, domaine, langue, URLs, labels et preuves d'image.
3. **Dédoublonnage** : conservation de la première ou de la dernière occurrence selon `--duplicate-policy`.
4. **Export** : typage avec pandas, tri stable, écriture atomique et création du manifeste.

Ce déroulé est découpé en fonctions ciblées : `map_and_validate_record()` traite une publication, `transform_batch()` isole un lot, `transform_inputs()` consolide les lots et `run_transformation()` pilote les sorties. Cette séparation permet de tester ou de faire évoluer une règle sans dupliquer le pipeline.

Un doublon est détecté par :

- la même valeur de `publication_id` ; ou
- le même couple `source_url` + `image_sha256`.

Chaque rejet contient la source, le fichier d'entrée, l'identifiant brut et la raison. Le manifeste enregistre les paramètres, les métriques, les hash des entrées et sorties, la version du schéma et le hash du pipeline.

Paramètres principaux :

Paramètre	Rôle
<code>--sources</code>	Sélectionner une ou plusieurs sources
<code>--input-mode all/latest</code>	Lire tous les lots ou le dernier de chaque source
<code>--duplicate-policy keep-first/keep-latest</code>	Choisir l'occurrence conservée
<code>--output-format parquet/jsonl/both</code>	Choisir les fichiers produits

5. Contrat final de 18 colonnes

Champ	Type final	Obligatoire	Rôle
<code>publication_id</code>	chaîne	Oui	Identifiant stable et PK logique
<code>source_name</code>	chaîne	Oui	Source d'acquisition
<code>source_domain</code>	chaîne	Oui	Domaine éditorial ou cible
<code>source_url</code>	URL HTTP(S)	Oui	Traçabilité de la publication
<code>title</code>	chaîne	Conditionnel	Texte court ; titre ou texte obligatoire
<code>text</code>	texte long	Conditionnel	Contenu destiné au NLP
<code>image_url</code>	URL HTTP(S)	Oui	URL distante de l'image liée
<code>image_path</code>	chemin	Oui	Image locale validée
<code>image_size_bytes</code>	entier	Oui	Contrôle du fichier vide ou modifié
<code>image_sha256</code>	chaîne hexadécimale	Oui	Intégrité et dédoublonnage
<code>image_provenance_status</code>	chaîne	Oui	État de la preuve URL-image
<code>published_at</code>	date-heure UTC	Oui	Date de publication
<code>language</code>	ISO 639-1	Oui	fr, en, etc.

Champ	Type final	Obligatoire	Rôle
author	chaîne	Non	Métadonnée éditoriale
source_label_raw	chaîne	Non	Label original non converti
source_label_scheme	chaîne	Non	Système donnant son sens au label
label_provenance	chaîne	Non	Auteur ou méthode de l'annotation
collected_at	date-heure UTC	Oui	Date de collecte du lot

Génération de publication_id

Source	Règle
NewsData.io	newsdata_ + article_id fourni par l'API
PolitiFact	politifact_ + 16 premiers caractères du SHA-256 de l'ID RSS ou de l'URL
Fakeddit	fakeddit_ + ID Reddit fourni dans le TSV
The Conversation	theconversation_ + ID numérique de l'article ; à défaut, 16 caractères du SHA-256 de l'URL

Le préfixe évite les collisions entre deux sources qui utiliseraient le même identifiant natif.

6. Schéma conceptuel pour la phase Load

Le contrat final contient une seule entité. Chaque ligne regroupe une publication, son texte, son image, ses métadonnées et son éventuel label. Le manifeste de transformation conserve séparément les informations d'exécution du pipeline.

PUBLICATION_MULTIMODALE		
string	publication_id	PK
string	source_name	
string	source_domain	
string	source_url	
string	title	
text	text	
string	image_url	
string	image_path	
integer	image_size_bytes	
string	image_sha256	
string	image_provenance_status	
datetime	published_at	
string	language	
string	author	
string	source_label_raw	
string	source_label_scheme	
string	label_provenance	
datetime	collected_at	

Lecture du schéma

PUBLICATION_MULTIMODALE représente une publication complète et directement exploitable :

- publication_id est la **PK** et identifie la ligne ;
- title et text représentent la partie textuelle ;
- les cinq champs image_* représentent l'image associée et sa preuve d'intégrité ;
- les champs source_*, author, published_at, language et collected_at assurent la traçabilité ;
- les trois champs de label restent facultatifs et indissociables.

Aucune **FK** n'est nécessaire puisqu'il n'existe qu'une table. Le lien texte-image est garanti par leur présence sur la même ligne et par les contrôles effectués pendant Transform.

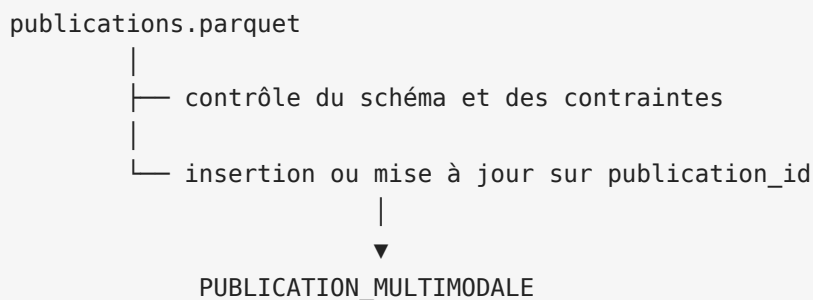
7. Contraintes et chargement

Contraintes d'intégrité

- `publication_id` est unique et non nul ;
- `source_name` et `source_domain` sont non nuls ;
- `title` ou `text` doit être renseigné ;
- `source_url` et `image_url` sont des URLs HTTP(S) ;
- `image_path` désigne un fichier local valide ;
- `image_size_bytes` > 0 ;
- `image_sha256` contient 64 caractères hexadécimaux ;
- `image_provenance_status` indique comment l'association URL-image a été vérifiée ;
- `published_at` et `collected_at` sont des dates UTC ;
- `language` contient deux lettres minuscules ;
- les trois champs de label sont tous présents ou tous absents ;
- le couple `source_url` + `image_sha256` est unique après dédoublement.

Chargement réalisé

Le DAG charge le Parquet dans la table `PUBLICATION_MULTIMODALE`, qui possède les mêmes 18 colonnes :



Le manifeste reste séparé comme fichier d'audit du pipeline ; il n'est pas mélangé aux publications.

8. État actuel et lecture correcte des résultats

Le dossier `data/raw/` contient actuellement **21 lots et 1 707 occurrences brutes** réparties entre les quatre sources. Chaque occurrence possède une preuve complète reliant l'URL source à l'image locale.

Le dernier manifeste `data/processed/transformation_manifest.json` correspond à une exécution consolidée des quatre sources :

- 21 lots et 1 707 occurrences lus ;
- 1 077 publications uniques exportées ;
- 630 doublons écartés ;
- 0 rejet de validation ;
- schéma 1.2 et 18 colonnes.

Le Parquet actuel est donc la sortie consolidée et validée des quatre sources. Il peut être reconstruit avec :

```
uv run python scripts/transform_data.py --sources newsdata politifact fakeddit theconversation --input-mode all --output-format both
```